

BUG-001 · Local Custom Chat-Completions Server Has No Embeddings Endpoint — OpenAIEmbeddings 404s Regardless of Config

Field	Detail
Issue	OpenAIEmbeddings pointed at the project's local CUSTOM_API_BASE server 404s on every embedding call, no matter what API key or model name is supplied
Found Date	2026-07-08
Status	Closed
Severity	HIGH
File	3_rag_v1.py
Description	After wiring OpenAIEmbeddings to the local server's CUSTOM_API_BASE/CUSTOM_API_KEY (mirroring the existing pattern used for chat completions in llm_setup.py), FAISS.from_documents(...) failed with openai.NotFoundError: Error code: 404 - {'detail': 'Not Found'}. Direct inspection of the server's /openapi.json confirmed it only exposes /v1/chat/completions, /v1/completions, and /health — it is a "TGI Proxy" with no /v1/embeddings route at all, so the failure is architectural, not a config or model-name mistake.
Root Cause	The local inference server only serves chat/completions traffic; embeddings were assumed to be available on the same endpoint by analogy with OpenAI's API, which serves both from one base URL.
Solution	Switched the embeddings client to langchain_huggingface.HuggingFaceEndpointEmbeddings (sentence-transformers/all-MiniLM-L6-v2, provider="hf-inference") using the project's existing HF_TOKEN, routed through the Hugging Face Inference Providers router instead of the local server. Confirmed working standalone (2 texts → 384-dim vectors) before wiring into 3_rag_v1.py. See ADR-001.
Date Resolved	2026-07-08

BUG-002 · with_structured_output Incompatible With Local TGI Proxy — Breaks 5_langgraph.py's Evaluation Nodes

Field	Detail
Issue	ChatOpenAI.with_structured_output(...) against the local CUSTOM_API_BASE server either 500s or returns tool-call arguments in a shape LangChain's AIMessage validation rejects
Found Date	2026-07-09
Status	Closed
Severity	MEDIUM
File	5_langgraph.py
Description	While migrating 5_langgraph.py off OpenAI (ADR-003), the default with_structured_output(EvaluationSchema) call (which uses the OpenAI beta.chat.completions.parse / json_schema response format) failed with openai.InternalServerError: Internal Server Error. Explicitly testing method="json_mode" also 500'd. Explicitly testing method="function_calling" got the model to actually produce the right tool call (correct score/feedback values), but LangChain then raised 1 validation error for AIMessage — invalid_tool_calls.0.args: Input should be a valid string [type=string_type, input_value={'score': 8, 'feedback': ...}, input_type=dict] — the proxy returns the tool call's arguments field as an already-parsed JSON object instead of a JSON-encoded string, which is what OpenAI's real API always returns and what langchain-openai assumes.
Root Cause	The local TGI Proxy's OpenAI-compatibility layer doesn't faithfully replicate OpenAI's tool-calling wire format (arguments as a string) or support the json_schema/json_mode response-format parameter, both of which with_structured_output depends on depending on method.
Solution	Replaced with_structured_output with a plain-prompt approach: PydanticOutputParser(pydantic_object=EvaluationSchema).get_format_instructions() appended to the prompt, llm.invoke(prompt).content parsed via parser.parse(...). Verified standalone against the local server (correctly parsed feedback/score from a markdown-fenced JSON response with trailing prose) before wiring into 5_langgraph.py's evaluate_dimension() helper. This approach doesn't depend on

	tool-calling support at all, so it works against any plain chat-completions endpoint.
Date Resolved	2026-07-09

BUG-003 · Weatherstack API Key In 4_agent.py Has Hit Its Monthly Usage Limit

Field	Detail
Issue	4_agent.py's get_weather_data tool always returns {'success': False, 'error': {'code': 104, 'type': 'usage_limit_reached', ...}}, causing the ReAct agent to loop on retries until it hits its iteration limit
Found Date	2026-07-09
Status	Open
Severity	LOW
File	4_agent.py
Description	Running 4_agent.py (after migrating its LLM off OpenAI, ADR-003) with the sample query "What is the current temp of gurgaon" showed the ReAct Thought/Action/Observation loop working correctly against the local model, but every get_weather_data call returned weatherstack's usage_limit_reached error (its free-tier monthly cap). The agent kept retrying the same failing tool call until AgentExecutor's max_iterations=5 cut it off, returning "Agent stopped due to iteration limit or time limit." instead of an answer. This is unrelated to the OpenAI-removal migration — the weatherstack key is hardcoded directly in the tool function and was already at its cap.
Root Cause	Weatherstack free-tier API key (hardcoded in get_weather_data's URL) has exhausted its monthly request quota; separately, the agent/prompt has no graceful handling for a tool that fails every time (it just repeats the same action).
Solution	Not fixed. Would need either a fresh weatherstack key / different weather API, or prompt/tool-error handling so the agent gives up and reports the failure after one bad observation instead of looping.
Date Resolved	—